FLIGHT MANAGEMENT SYSTEM

Comprehensive System Design & Architecture Document

Concurrency-Safe Booking • Dual-Writer Governance • RAG-Assisted Support

Stack: FastAPI · Supabase / Neon Postgres · n8n · Pinecone · OpenAI API

PREPARED FOR PROJECT DOCUMENTATION
Version 1.0

01 The Core Problem

Modern flight management systems must operate correctly under high-concurrency conditions, where hundreds of users may attempt to claim the same seat within milliseconds of one another. Three risks drive the design decisions in this document.

The Overselling Dilemma

Overselling prevention must be atomic, so it lives entirely inside FastAPI against Postgres. If two users press “Book” on the last available Economy seat at the same instant, the database must serialize these requests so that only one succeeds.

The Dual-Writer Conflict

The architecture uses a shared database with two independent writers — FastAPI and n8n. The central risk is that these systems corrupt each other's data: for example, a gate agent manually reassigning a seat at the same moment an automated waitlist workflow tries to claim that same seat.

Data Integrity Bypass

Because n8n connects directly to Postgres, it completely bypasses any business logic or validation implemented in the Python application layer. Any invariant that matters must therefore be enforced by the database itself, not just by application code.

02 The Architectural Solution

To resolve the concurrency and integrity risks above, the system is built around strict boundary constraints and database-level enforcement rather than relying on application discipline alone.

- **Separation of Concerns —** FastAPI is the only write path for live booking and admin operations, writing directly to Supabase Postgres. n8n reads and writes the same Postgres tables independently for scheduled and automated work, with no runtime dependency on FastAPI and no shared code contract with it.
- **Table Ownership —** Every table has exactly one system designated as its sole writer, eliminating ambiguity about which service is authorized to modify which data.
- **Database-Level Enforcement —** Because n8n bypasses FastAPI's validation layer, data integrity is guaranteed using Postgres-level invariants: CHECK constraints, foreign keys, and enums, so that no writer — regardless of path — can violate core business rules.

*For Level-0 / Level-1 Data Flow Diagrams and UML Use Case models, the primary actor for live transactions is the **User / Admin interacting with FastAPI**, while the primary actor for asynchronous work is the **n8n schedule trigger operating directly on the Postgres data store**.*

03 Technology Stack & Key Concepts

Technology	Role in Architecture	Core Responsibility
Neon / Supabase Postgres	Core Ledger	Serves as the single ledger of truth for flights, seats, and bookings. Manages atomic operations and row-level locks.
FastAPI (Python)	Live Transaction Engine	Handles live, request-triggered, transactional writes. Provides typed request/response schemas and validation for every booking-affecting endpoint.
n8n (Node.js)	Automation Engine	Manages scheduled and background, direct-to-Postgres operations.
Pinecone	Vector Database	Stores document embeddings for the AI support system. Maintains an index-maintenance job for policy docs and fraud-pattern embeddings.
OpenAI API	LLM Engine	Generates embeddings for policy documents and drafts context-aware responses to customer support inquiries.

04 Database Schema: The Single Ledger of Truth

The schema is designed to enforce business logic directly at the database layer, so that invariants hold no matter which service performs the write.

- **Enums —** *seat_class* (First, Business, Economy) and *booking_status* (Held, Confirmed, Cancelled).
- **Flights Table —** Enforces duplicate flight-number detection for the same day and route via a UNIQUE constraint on flight number and departure time.
- **Flight Seat Classes Table —** Includes a CHECK (total_allocated > 0) constraint to reject seat-class allocations with negative or zero values.
- **Bookings Table —** Tracks booking status and relies on a hold_expires_at timestamp to manage temporary reservations.

05 Live Transactional Engine: FastAPI

FastAPI manages all synchronous, user-initiated actions and ensures data consistency before a write ever reaches the database.

Admin & Flight Management

FastAPI owns all flight-creation and inventory-defining writes; n8n has no direct role here.

- **Flight Creation —** Exposes an admin endpoint to create a new flight, capturing origin, destination, departure/arrival datetime, and flight number.
- **Capacity Enforcement —** The endpoint strictly validates that seat-class totals sum exactly to the declared aircraft capacity, and explicitly rejects seat-class allocations that are negative, zero, or non-integer.

Search & Booking Flow

Search and pricing logic must remain transactionally consistent with live inventory at all times.

- **Real-Time Search —** A search endpoint returns available seats per class for a given route and date.
- **The Hold Mechanism —** When a user selects a seat, the system creates a temporary seat hold during checkout, with an expiry if payment is not completed.
- **Atomic Transactions —** The hold is executed via an atomic seat-class decrement, preventing two simultaneous bookings from selling the same last seat. This atomic UPDATE guarantees inventory is decremented safely without race conditions.

06 Scheduled Automations: n8n

Time-based work with no live user request in the loop is n8n's core responsibility. Decoupling automations from the live API keeps the web server free of background-timer management, fitting naturally into an incremental, agile delivery model.

Customer Notifications

- **Check-in Alerts —** Dispatches a check-in reminder email that is timezone-correct for both origin and destination.
- **Communication Gateway —** Uses the Gmail node integration for all customer-facing scheduled notifications.

Revenue & Operations

- **Reporting —** Generates daily and weekly operations reporting — load factor, revenue per flight — pulled directly from Postgres.
- **Refund Management —** Triggers an escalation notification if a refund remains unresolved after N days.

Waitlist Operations

Joining a waitlist is a live booking action handled by FastAPI; promoting people off the waitlist is background polling handled by n8n.

- **Promotion Logic —** A scheduled job detects freed-up seats and promotes the next waitlisted passenger.
- **Concurrency Control —** Row-locking ensures a gate-agent action and an n8n promotion run can never assign the same freed seat twice (e.g., using `SELECT ... FOR UPDATE SKIP LOCKED`).

Fraud Detection

Kept entirely within n8n scheduled jobs reading directly from Postgres and Pinecone, with no FastAPI involvement.

- **Real-Time Review —** A scheduled bot / mass-booking fraud-scoring job scans recent bookings.
- **Historical Analysis —** A batch review job scans historical bookings for fraud patterns missed in real time.

07 AI Support & RAG Pipeline (Pinecone + n8n)

To support customer service agents, the architecture includes an AI pipeline designed to interpret complex airline policies accurately and safely.

- **Vector Ingestion —** A scheduled ingestion pipeline embeds updated policy documents into Pinecone, chunking internal documents and converting them into embeddings.
- **Context-Aware Responses —** When a support ticket arrives, the workflow ensures the RAG-drafted answer to policy questions reflects the booking's actual fare rule, not a generic match — the LLM is given both the vector search results and the user's specific database record.
- **Safety Protocols —** To prevent hallucinated responses from reaching customers, the system mandates a human approval gate before any RAG-drafted answer is sent via Gmail.

This document summarizes the target architecture for the Flight Management System: a single Postgres ledger of truth, a FastAPI live-transaction boundary enforcing business rules on every user-facing write, an independent n8n automation layer constrained by table-ownership rules and database-level invariants, and a human-supervised RAG pipeline for policy-accurate customer support.